1. **Accessing the model from the TE Model:**
Usage of SELinux Type Enforcement (TE) model over the robotic systems offers a robust Mandatory Access Control (MAC) layer that does not allow the compromised processes to travel across the system laterally. Within a robot, in which a navigation module may not in any way be expected to be reading camera data, or even be able to have access to the internet, TE makes each process (subject) capable only of interaction with specific resources (objects) which it is explicitly allowed to utilise. SELinux Type Enforcement (TE) model may also be used to better enforce the security of robotic systems, by strictly regulating which processes can access specific files, devices, memory space, network interfaces, and other system resources. In TE, each process-like subject is assigned a domain, and each object, like a file, socket, device, or directory, is assigned a type. The security policy does not permit access to a domain until the domain is explicitly permitted by the security policy to execute some operations on a type. This is based on the principle of least privilege, which is of particular concern to robotic systems since robots frequently engage with sensors, actuators, cameras, motors, communication modules, and safety-critical control software.

As an example, consider a robot with three key elements, a vision process, a navigation process, and a motor-control process. The vision process may be running in a domain like vision_t, the navigation service in navigation t, and the motor controller in motor_t. CameraDevice may be called camera_device_t, map map_data_t, and motor motor_device_t. The policy might permit vision to read a camera device, permit navigation to read map data, communicate with vision, and permit motor to write to the motor device only. And, simultaneously, vision would not be directly allowed to access the motor devices, and the motor would not be provided the privilege to alter map files. By doing so, TE minimizes the harm that might otherwise occur when the exploitation of a single software component occurs. The structure of the Access Control Rule.

A typical rule in SELinux has the following syntax:
allow source type target type: class permissions;

A Lidar Driver and a Log File would look like:

Source Type: lidar_t (The process type)

Target Type: lidar log t (The file type)

Class: file (The object kind)

Authors, write, append (The actions allowed)

The main advantages of Robotic Security include:
Sandboxing Modules: In case the web-based dashboard of a robot is compromised, the dashboarding domain will not allow unauthorized requests to the motor controller domain of the motor controller of the robot.

Root Containment: SELinux TE (not like traditional Linux (DAC)) can deny an attacker access to sensitive hardware ports or configuration files, even after gaining the root privilege.

Hardware Isolation: You may tag certain /dev/ nodes (such as /dev/ttyUSB0) in such a way that only the sensor driver in question can open them.
The manual design of TE policies is difficult, prone to mistakes, and demands thorough knowledge of the system, although the use of AI assistants can greatly simplify the task, especially the use of large language models. They allow natural language representations to generate policy: just describe it, e.g. to an engineer as the camera processing node must be capable of reading the camera device and writing to a common memory location, but it must not be allowed to read or write to the motor device, and the AI assistant translates the description into specific SELinux rules, including appropriate type definitions. Also, AI can be used to assist policy refinement in automated mode by analyzing system logs during development, like audit.log denials, as well as suggest extra rules to permit legal activities and deny unacceptable ones, which minimizes the number of iterations required to write the policy. To begin with, AI can be used to determine the resources that any robotic process actually requires with respect to the description of the system, its software architecture, and the anticipated workflow. Second, AI has the capability of proposing the first-time SELinux policy rules based on least-privilege constraints, and the developers can be much less active than they would be unable to do by hand. Third, AI would be able to look over draft policies and highlight instances of permissions being too broad, inconsistent, or unnecessary. To be specific, in case a developer forgets to regulate the access of a camera process to actuator devices, an AI-assistant can point out that as an insecure policy decision. AI can also be used to translate high-level security requirements into low-level SELinux allow rules, type definitions, and domain transitions.

Nevertheless, AI must be considered more of an auxiliary, but not an ultimate power. The robotic systems being safety sensitive means that even now, human specialists have to check the rules, test the policies, and ensure that the created permissions correspond with the operational demands. The AI assistant can accelerate the process of policy creation, clarify the mistakes in policy formulation, and troubleshoot the case of denials, yet thoughtless AI proposals can introduce unsafe access points in case they are adopted without reconsideration.

2. Robot Operating System

Robot Operating System (ROS) is an open-source system architecture to develop robot applications. Although this might seem like a full operating system, ROS in fact is middleware that is used on top of an operating system, like Linux. It assists the various components of a robot to communicate in an ordered fashion. A robot in ROS is typically subdivided into tiny software modules known as nodes. Each node is responsible to do a particular task, like reading sensor data, movement control, camera image processing, or decision-making. These nodes interact by exchanging messages over topics, asking actions over services, or organizing more elaborate goals over actions. This makes robotic systems simple to build, test, and maintain because of this modular design.

As an example, a ROS node might gather data with a camera, another node might analyze that image and identify any obstacles, and another node might determine how to move the robot. The movement

instructions would then be sent to a motor-control node which would send the commands to the robot wheels or joints. Since all the nodes are assigned a particular task, ROS gives developers the chance to create complex robots out of smaller and reusable components. This also leads to better reliability, as it is frequently only necessary to update or repair a single component of the system, instead of redesigning the entire robot. The autonomous courier Unit-A42 is traveling along the high-tech corridors of a logistical center in the year 2027, its brain structured as a modular network of ROS nodes. Since the navigationnode is publishing velocity orders to the /cmdvel topic, the Linux kernel beneath implements an ironclad airtight control on security by incorporating SELinux Type Enforcement. The attack quickly reaches a dead end when a rogue program is trying to steal the visionnode to scavenge sensitive mapping information in the root directory of the system. Since the node is restricted to the visiont domain, policy specifically prohibits the node access to any resource that is not in its sandbox, including the /etc configuration files. The robot is stable even having a compromised process; the sensornode keeps Lidar data on /dev/ttyUSB0, and the commsnode remains isolated within an encrypted network stack. A42 goes on its way, a demonstration of a day when the intelligent robots are inseparable with strict kernel-level access control.

text Copy Downloa

```text
allow camera_t camera_device_t : chr_file { read ioctl };
allow detector_t shared_memory_t : shm { read write };
allow nav_t motor_controller_t : unix_stream_socket { connect write };
allow motor_t motor_device_t : chr_file { read write ioctl };
# No rule → no access: detector_t cannot touch motor_device_t, etc.
```

Source: DeepSeek

3. Task 1 - Describing My Robot System

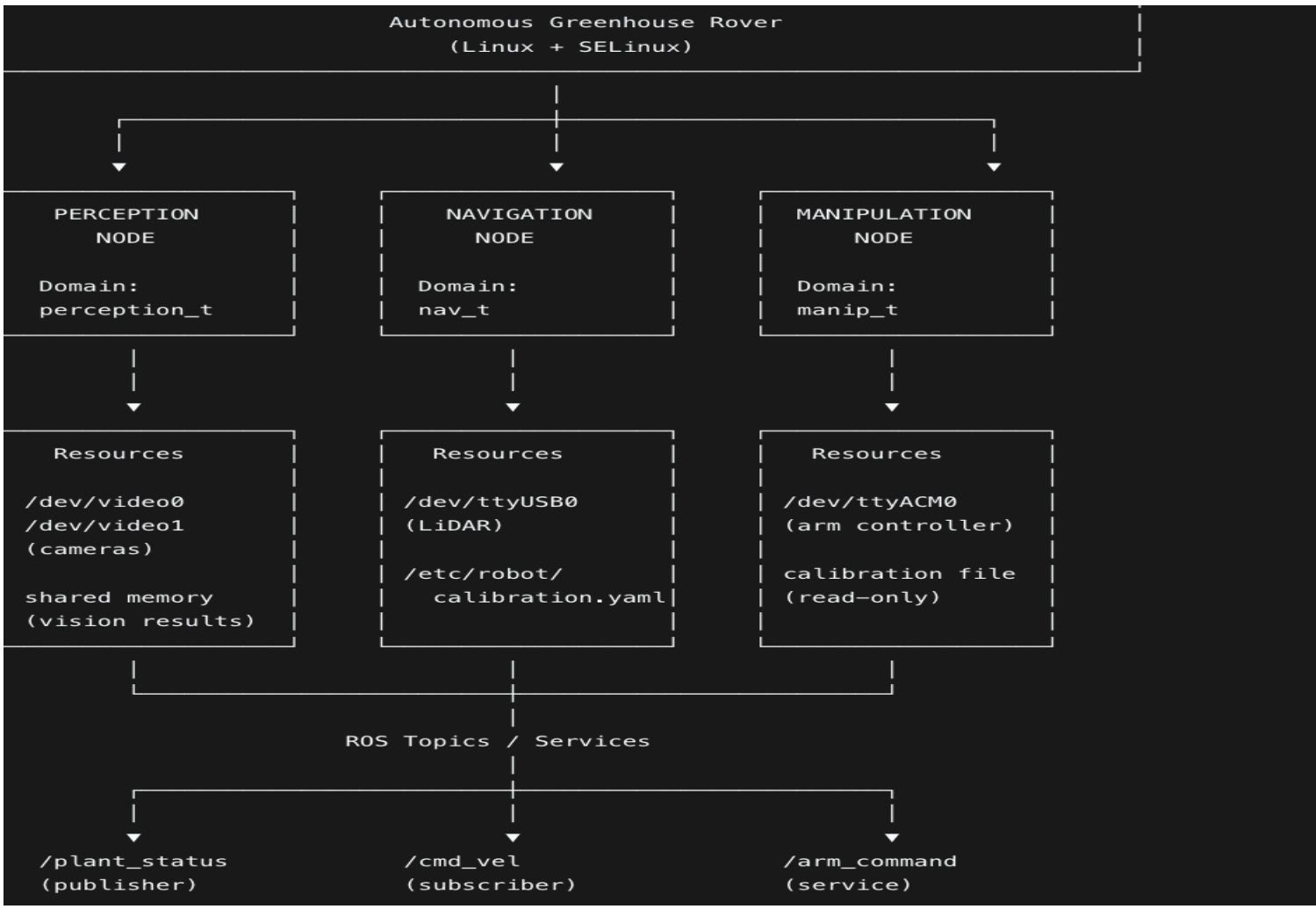
The robot system, an autonomous greenhouse robotic system inspection and maintenance is a design that is used to monitor crop health, pest identification, and selective watering. It is aimed at eliminating human labor and maximizing the use of resources in large-scale hydroponic farms. The rover manages the rows of plants through the combination of the wheel encoders and the visual odometry and a robotic arm is placed to perform accurate interventions.

The system is based on ROS 2 (Humble) operating on embedded Linux computer. The main functions are to do with three ROS nodes. The perception node is subscribing to images streams of a multispectral camera and thermal camera. It uses computer vision models to detect stressed plants, ascertain pest infestations, and compute fruit ripeness. The navigation node takes odometry and LiDAR data and localizes the rover, plans collision-free routes, and publishes velocity commands to the differential drive. The manipulation node is used to operate a six-degree-of-freedom arm; gripping or spraying is done using low-level hardware interfaces and is fed target positioning information by the perception node.

The camera devices (/dev/video0, /dev/video1), LiDAR serial interface (/dev/ttyUSB0), and motor controller device (/dev/ttyACM0) are considered to be important system resources. Other important files

that the rover utilizes are a calibration parameter file (/etc/robot/calibration.yaml) and a cryptographic key file to provide secure telemetry uploads (/etc/robot/cloud_key.pem). The network access will be necessary to broadcast aggregated data to a central farm-management system via TLS-encrypted MQTT.

One such security threat is that a hacked perception node, such as a maliciously constructed image that exploits a vulnerability in the vision library may be employed to affect the physical world. Using the CIA triad, integrity and availability are the main targets of the threat. An attacker would be able to exploit the coordinates that would be disseminated to the manipulation node in the manner that the arm sprays water randomly, wastage of resources, or even destruction of plants. Or, the attacker may inundate the ROS topic with high frequency velocity commands and override the navigation node, making the rover run into buildings and hence breaking availability. This is a tampering (modifying command messages) and denial of service (flooding or crashing the navigation stack) attack in terms of STRIDE. Since ROS 2 is not designed with fine-grained user-to-user node permissions, one vulnerable node can easily compromise other nodes unless the use of access controls like SELinux is made mandatory to define the domain of each node and limit the topics and hardware devices to which it can be connected.



Source: ChatGPT

4. Task 2: Identifying Subjects and Objects

<u>Subjects (domains):</u>

perception_t - ROS process can capture camera images.
navigation_t - ROS node that manages locomotion.
manipulation_t - ROS node which runs the robotic arm.

<u>Access per subject to objects (resources):</u>

Perception_t:
/dev/video0, /dev/video1 (cameras) read, ioctl.
shared memory segment (vision results) -read write.
Not access to motor or arm devices.

Navigation_t:
/dev/ttyUSB0 (LiDAR) – read
calibration.yaml (calibration file) -read only etc/robot/calibration.yaml.
Socket /cmd_vel topic write
No access to camera devices

Manipulation_t:
dev/ttyACM0 (arm controller) -r, w.
et.// robot/calibration.yaml – read.
/arm command service, read, write.
None to LiDAR or motor devices.

5. Task 3: Working with AI Assistant to generate SELinux Rules:

From the AI tool: Deepseek, these are the rules generated below:

Task 3 — Generate SELinux TE Rules

Based on the subjects and objects identified in Task 2, here are three SELinux Type Enforcemen rules:

1. **perception_t** accessing camera devices
   `allow perception_t camera_device_t : chr_file { read ioctl };`
2. **navigation_t** accessing LiDAR device
   `allow navigation_t lidar_device_t : chr_file { read };`
3. **manipulation_t** accessing arm controller device
   `allow manipulation_t arm_device_t : chr_file { read write };`

These rules enforce least privilege by granting each ROS node only the permissions it needs to access its required hardware resources, while implicitly denying all other accesses.

Source : Deepseek

6. Task 4: Evaluating the AI Inputs

The three SELinux rules that have been formed by the AI assistant provide certain access between a subject domain and an object type. In the case of the first rule, which is, permission perception_t camera_device_t: chr file read IOctl, the AI permits the perception node to read and access control operations on the camera devices. I concur with this rule as the perception node is a valid user of camera data in order to accomplish its vision functions; it could not use an ioctl in order to set up resolution or frame rate. Yet the rule ought not to be permitted in its present form-it would be wiser to limit it to a particular device node (e.g., /dev/video0) instead of an unspecialized camera_device_t, which may have more than one camera in it, and the permission to do IOCs should be restricted to safe operations.

In the second rule, which is allow navigation t lidar t device t: chrfile read, the AI accepts read access to the LiDAR device. I consider it necessary to make navigation work, but this rule should not be permitted in such a loose form since it does not specify the type of device node or the type of interface; a more precise type, such as lidar_serial_t, should be used, and perhaps more permissions are required, such as ioctl, if the driver requires configuration. In addition, the rule implicitly permits the navigation node to read the LiDAR indefinitely, but in reality, the node may need to access the device only on restart; this may be made mandatory through a domain transition or by a helper process.

In the case of the third rule, manipulation_t arm_device_t chr_file read write, the AI permits the read and write access of the arm controller device. I concur that the manipulation node needs to command the arm and status feedback; a read is frequently required. However, this rule cannot be implemented without further restrictions: the type of device must be a particular arm_serial_t, and the permission set may be reduced to just read and write but may not to be able to use the ioctl command unless it is necessary. Additionally, the policy must not allow any other node besides the manipulation node to access this vital hardware, and this rule is already followed by omission. All in all, the rules that the AI suggests are a good starting point, although all of them will have to be narrowed down to more concrete types and least-privileged permissions to reflect the needs of real-life security considerations.

7. Task 5: Creating one access request that should not be allowed

The access request that shall be rejected is:
allow perception_t arm_device_t: chr_file { write };

This would also allow the perception node to command the robotic arm controller outright, as the perception node is only charged with camera processing. This access is a breach of the principle of least privilege since the perception node does not require any legitimate access to physical actuators; its job is restricted to the publication of plant health coordinates. With this rule permissible, an attacker could compromise the perception node (e.g., a buffer overflow in the vision library) to send arbitrary commands to the arm and cause damage to plants, water overflow, or harm to personnel near the machine. The system is highly isolated by ensuring that perception and manipulation are in different SELinux domains and that they cannot access hardware devices across domains. Rejecting this request will guarantee safety and security because even when the perception node is compromised, the arm will be under the sole control of the manipulation node.